Projeto React

Prof. Me. Deivison S. Takatu

deivison.takatu@edu.senai.br

SENAI

REVISÃO JAVASCRIPT

Javascript

- O JavaScript pode ser inserido diretamente no corpo do HTML usando a tag <script>, permitindo executar funções simples sem a necessidade de arquivos externos.
- Outra opção é usar arquivos externos, vinculados com o atributo src da tag <script>, prática recomendada em projetos maiores por facilitar organização e manutenção do código.
- Também é possível incluir JavaScript nos atributos de eventos, como onclick e onblur, que tornam a página mais interativa ao responder às ações do usuário.

Javascript

O JavaScript oferece uma ampla variedade de recursos que permitem controlar praticamente todos os aspectos dinâmicos de uma página web. Além dos conceitos básicos abordados em aula, existem inúmeras funcionalidades e APIs especializadas para criar interatividade avançada, manipular o DOM, desenvolver animações, aplicar efeitos visuais, consumir dados de servidores e construir aplicações web modernas e responsivas.

Referência: developer.mozilla.org

Javascript

```
<!-- Corpo da tag SCRIPT -->
<script>
    var teste = "Ola";
</script>

<!-- Atributo src -->
<script src="script.js"></script>

<!-- Atributos de eventos -->
<button onclick="alert('Ola')">Teste</button>
```

Fonte: Elaboração Própria.

Javascript - Variáveis

Em muitas linguagens de programação, é obrigatório declarar todas as variáveis no início do código, mas no JavaScript isso não é necessário. A linguagem permite que variáveis sejam declaradas em qualquer ponto do script, oferecendo maior flexibilidade para o desenvolvedor.

Outro detalhe importante é que os nomes das variáveis em JavaScript são case sensitive, ou seja, diferenciam letras maiúsculas de minúsculas. Dessa forma, variáveis como ***nome***, ***Nome*** e ***NOME*** seriam tratadas como identificadores distintos.

Javascript - Variáveis

Em JavaScript, var possui escopo de função, ou seja, uma variável declarada com var é acessível em toda a função em que foi criada, mesmo dentro de if, for ou while, e se declarada fora de qualquer função, é global. Já let possui escopo de bloco, existindo apenas dentro do bloco { } em que foi declarado, o que torna seu uso mais restrito e previsível.

```
var nome = "Ana";                 // String
let idade = 25;                   // Number
const estudante = true;           // Boolean
let frutas = ["Maçã", "Banana"]; // Array
let pessoa = { nome: "Carlos", idade: 30 }; // Objeto
let endereco = null;              // Null
let telefone;                     // Undefined

console.log(nome, idade, estudante, frutas, pessoa, endereco, telefone);
```

Fonte: Elaboração Própria.

Javascript - Variáveis

```
console.log("=== 01 — Variáveis e Tipos ===");

const nome = "João";
let idade = 20;
const estudante = true;

console.log(nome);
console.log(idade);
console.log(estudante);

console.log(typeof nome);
console.log(typeof idade);
console.log(typeof estudante);

console.log(`Olá, ${nome}! Você tem ${idade} anos.`);
```

Tipos de dados em JavaScript

String - "Olá"

Number - 10, 3.14

Boolean - true, false

Undefined - valor não definido

Null - ausência intencional de valor

Object - { nome: "Ana", idade: 20 }

Array - [10, 20, 30]

Symbol - identificador único

BigInt - números inteiros muito grandes

typeof - verifica o tipo de um valor

*O JavaScript identifica automaticamente o tipo de dado armazenado

Operadores em JavaScript

Aritméticos

+ → soma dois valores

- → subtrai um valor de outro

* → multiplica valores

/ → divide um valor por outro

% → retorna o resto da divisão

** → realiza uma exponenciação

++ → incrementa o valor em 1

-- → decrementa o valor em 1

Comparação

== → compara os valores, permitindo conversão de tipo

=== → compara valor e tipo

!= → verifica se os valores são diferentes

!== → verifica se valor ou tipo são diferentes

> → maior que

< → menor que

>= → maior ou igual a

Operadores em JavaScript

Lógicos

&& → E: todas as condições devem ser verdadeiras

|| → OU: pelo menos uma condição deve ser verdadeira

! → NÃO: inverte o resultado lógico

Atribuição

= → atribui um valor

+= → soma e atribui

-= → subtrai e atribui

*= → multiplica e atribui

/= → divide e atribui

%= → calcula o resto e atribui

Operadores em JavaScript

```
console.log("=== 02 — Operadores ===");

const nota1 = 8;
const nota2 = 7;

const media = (nota1 + nota2) / 2;

console.log("Média:", media);
console.log("Aprovado:", media >= 7);
console.log("Nota1 === Nota2:", nota1 === nota2);
```

Javascript - Condicionais

Em JavaScript, uma condicional é uma estrutura de controle que permite executar diferentes blocos de código dependendo de uma condição booleana. A mais comum é o if, que avalia uma expressão e executa o bloco de código associado se a condição for verdadeira, podendo ser complementado por else para executar outro bloco caso a condição seja falsa, e por else if para testar múltiplas condições em sequência.

```
var idade = 20;

if (idade >= 18) {
  var mensagem = "Você é maior de idade.";
} else {
  var mensagem = "Você é menor de idade.";
}

console.log(mensagem);
```

Fonte: Elaboração Própria.

Javascript - Condicionais

```
console.log("=== 03 — Condicionais ===");

const nota = 8;

if (nota >= 9) {
    console.log("Excelente");
} else if (nota >= 7) {
    console.log("Aprovado");
} else {
    console.log("Reprovado");
}

const resultado = nota >= 7 ? "Aprovado" : "Reprovado";
console.log(resultado);
```

Estruturas de repetição

- As estruturas de repetição permitem executar um bloco de código várias vezes, enquanto uma determinada condição for atendida ou durante uma sequência definida.
  - for → utilizado quando sabemos ou controlamos a quantidade de repetições.
  - while → repete enquanto uma condição for verdadeira.
  - do...while → executa o bloco pelo menos uma vez e depois verifica a condição.
  - break → interrompe a repetição imediatamente.
  - continue → interrompe a iteração atual e passa para a próxima.

Estruturas de repetição

```
console.log("=== 04 — Repetições ===");

for (let i = 1; i <= 10; i++) {
    console.log(i);
}

console.log("Pares:");

for (let i = 1; i <= 10; i++) {
    if (i % 2 === 0) {
        console.log(i);
    }
}
```

Javascript - Funções

Em JavaScript, funções são blocos de código que executam uma tarefa específica e podem retornar um valor. Elas permitem organizar o código e evitar repetição, recebendo parâmetros como entrada e devolvendo resultados com return.

Funções podem ser declaradas de forma tradicional (function nome() {}) ou como arrow functions (const nome = () => {}). Elas são usadas para manipular dados, arrays, elementos HTML e eventos, tornando o código mais modular, legível e fácil de manter.

Javascript - Funções

```
console.log("=== 05 – Funções ===");

function somar(a, b) {
    return a + b;
}

const resultado = somar(10, 20);

console.log("Resultado:", resultado);

const multiplicar = (a, b) => a * b;

console.log("Multiplicação:", multiplicar(5, 4));
```

Javascript - Exemplos de Funções

- **console.log()** – Mostra mensagens no console (útil para depuração).
- **alert() / prompt() / confirm()** – Interação básica com o usuário.
- **document.getElementById()** / document.querySelector() – Seleciona elementos do HTML.
- **.addEventListener()** – Adiciona eventos (cliques, scroll, input, etc.).
- **.innerHTML / .textContent** – Altera o conteúdo de elementos HTML.

Javascript - Arrays

- Arrays são estruturas utilizadas para armazenar vários valores em uma única variável.
- Os elementos são organizados em uma sequência e podem ser acessados por sua posição, chamada de índice.
- Exemplo:
  - const frutas = ["Maçã", "Banana", "Laranja"];

Javascript - Arrays

```
console.log("=== 06 — Arrays ===");

const notas = [7, 8, 5, 9, 10];

console.log("Notas:", notas);
console.log("Quantidade:", notas.length);

notas.forEach((nota, indice) => {
    console.log(indice, nota);
});

const aprovados = notas.filter(nota => nota >= 7);

console.log("Aprovados:", aprovados);
```

Objetos em JavaScript

- Objetos são estruturas utilizadas para armazenar informações relacionadas em propriedades e comportamentos em métodos.
  - Propriedades → armazenam dados do objeto.
  - Acesso → permite consultar ou alterar suas propriedades.
  - Métodos → funções associadas ao objeto.
  - this → referencia o próprio objeto dentro de seus métodos.

Objetos em JavaScript

```
console.log("=== 07 – Objetos ===");

const aluno = {
    nome: "Carlos",
    idade: 20,
    curso: "ADS",

    apresentar() {
        console.log(`Meu nome é ${this.nome} e curso ${this.curso}.`);
    }
};

console.log(aluno.nome);
console.log(aluno.curso);

aluno.apresentar();
```

Javascript - Exemplos de Código

```javascript
// Dark mode
const toggleBtn = document.getElementById('toggle-dark');
toggleBtn.addEventListener('click', () => {
  document.body.classList.toggle('dark');
});

// Botão voltar ao topo
const backToTop = document.getElementById('back-to-top');
window.addEventListener('scroll', () => {
  backToTop.style.display = window.scrollY > 200 ? 'block' : 'none';
});
backToTop.addEventListener('click', () => {
  window.scrollTo({ top: 0, behavior: 'smooth' });
});
```

Fonte: Elaboração Própria.

Javascript - Exemplos de Código

```javascript
// Efeito de digitação (typewriter)
const text = "Bem-vindo ao site de exemplos JavaScript!";
const typewriter = document.getElementById('typewriter');
let index = 0;
function type() {
  if (index < text.length) {
    typewriter.textContent += text.charAt(index);
    index++;
    setTimeout(type, 100);
  }
}
type();
```

Fonte: Elaboração Própria.

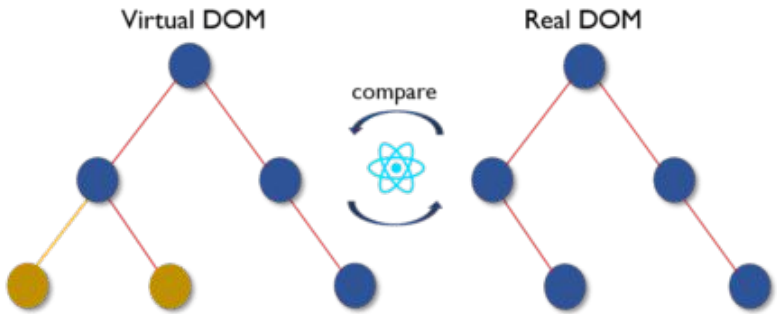
Javascript - Exemplos de Código

```javascript
// Pop-up (modal)
const modal = document.getElementById('modal');
document.getElementById('open-modal').addEventListener('click', () => {
  modal.style.display = 'block';
});
document.getElementById('close-modal').addEventListener('click', () => {
  modal.style.display = 'none';
});
window.addEventListener('click', (e) => {
  if (e.target === modal) modal.style.display = 'none';
});
```

Fonte: Elaboração Própria.

Definição de DOM (Document Object Model)

O DOM é uma representação em árvore da estrutura de uma página web, permitindo que o JavaScript altere seu conteúdo. O Virtual DOM (usado no React) é uma cópia mais rápida desse modelo. Quando algo muda, o React primeiro atualiza essa cópia, compara com o DOM real e só aplica as diferenças, tornando tudo mais rápido e eficiente.



Fonte: https://www.mindbowser.com

Selecionando elementos do DOM

- O JavaScript permite localizar elementos HTML no DOM para posteriormente consultar ou modificar seu conteúdo.
  - getElementById() → seleciona um elemento pelo id
  - querySelector() → seleciona o primeiro elemento que corresponde a um seletor CSS
  - querySelectorAll() → seleciona todos os elementos que correspondem ao seletor CSS

Alterando o DOM com JavaScript

- O JavaScript permite modificar elementos HTML diretamente pelo DOM, alterando seu conteúdo, valores e classes.
  - textContent → altera o texto de um elemento
  - value → altera o valor de campos de formulário
  - classList → adiciona, remove ou alterna classes CSS
  - createElement() → cria novos elementos HTML
  - appendChild() → adiciona elementos ao documento

Exemplo: JS + DOM

```
// Seleciona o botão pelo ID
const botao = document.querySelector("#botao");

// Seleciona o elemento que exibirá a mensagem
const texto = document.querySelector("#mensagem");

// Adiciona um evento de clique ao botão
botao.addEventListener("click", () => {

    // Altera o texto do elemento
    texto.textContent = "Olá! O conteúdo foi alterado.";
});
```

Atividade de Revisão

Link: https://wayground.com/

Material: https://github.com/deivisontakatu/aula-javascript



SENAI

Sumário

- JSX: sintaxe e expressões JavaScript
- Componentes: criação, composição e reutilização
- Props: passagem de dados entre componentes
- DOM e Virtual DOM: renderização e reconciliação
- Listas e Keys: renderização de coleções de dados
- Renderização condicional: controle de exibição da interface
- Atividade

Links Complementares

- Conceitos: https://github.com/deivisontakatu/aula-react
- Exemplo: https://github.com/deivisontakatu/projeto-react
- Aplicação: https://projeto-react-five-swart.vercel.app/

SENAI

JSX

SENAI

O que é JSX

- JSX é uma extensão de sintaxe para JavaScript.
- Permite escrever estruturas semelhantes a HTML dentro do JavaScript.
- É utilizado para descrever interfaces de componentes React.
- Combina a estrutura da interface com a lógica que produz seu conteúdo.

```
const titulo = <h1>Olá, React!</h1>;
```

SENAI

Por que utilizar JSX

- Facilita a leitura da estrutura da interface.
- Combina elementos visuais e expressões JavaScript.
- Organiza interfaces dentro de componentes.
- Relaciona dados ao conteúdo exibido.

Principal vantagem: simplificar a leitura e a manutenção das interfaces.

Expressões JavaScript

- É possível utilizar as chaves { } para inserir expressões JavaScript no JSX:

```
const nome = "João";
const saudacao = <h1>Olá, {nome}!</h1>;
const a = 10;
const b = 5;
const resultado = <p>Soma: {a + b}</p>;
```

SENAI

Diferenças entre JSX e HTML

- Utiliza className em vez de class.
- Utiliza htmlFor em vez de for.
- Elementos sem conteúdo devem ser fechados, como <img />.
- Componentes devem retornar um elemento raiz ou utilizar Fragment.
- Valores JavaScript são inseridos entre chaves {}.

SENAI

JSX e atributos

```
function LinkDocumentacao() {
  return (
    <a href="https://react.dev" className="link">
      Documentação do React
    </a>
  );
}
```

O componente retorna um link clicável para a documentação do React.

SENAI

Utilização do JSX

- Facilita a descrição das interfaces React.
- Permite inserir expressões JavaScript entre chaves {}.
- Possui convenções diferentes do HTML.
- Organiza interfaces em componentes reutilizáveis.
- É transformado em JavaScript antes da execução.

SENAI

COMPONENTES

SENAI

O que é um componente?

- Componentes são unidades independentes e reutilizáveis da interface.
- Em React, são normalmente funções JavaScript que retornam elementos React.
- Geralmente utilizam JSX para descrever a estrutura visual.
- Permitem dividir uma aplicação em partes menores e organizadas.

SENAI

O que é um componente?

```
function Cabecalho() {
  return (
    <header>
      <h1>React Lab</h1>
    </header>
  );
}
```

Na prática: uma loja virtual pode ter componentes como Cabecalho, Busca, Produto e Rodape.

Por que dividir uma interface?

- Uma página pode ter várias partes, como menu, busca, cartões e rodapé.
- Colocar tudo em um único componente dificulta a leitura.
- Dividir a página facilita a organização do código.
- Cada parte pode ser reutilizada em outras páginas.

Exemplo: separar o cabeçalho, os produtos e o rodapé de uma loja virtual.

SENAI

Como utilizar componentes no JSX?

- Crie uma função com nome iniciado em letra maiúscula.
- Use return para definir o que será exibido.
- Utilize <NomeDoComponente /> para exibir o componente.
- Combine componentes para montar uma página.

SENAI

Como utilizar componentes no JSX?

```
function Cabecalho() {
  return <h1>Catálogo React</h1>;
}
function App() {
  return (
    <main>
      <Cabecalho />
      <p>Explore os principais conceitos.</p>
    </main>
  );
}
```

SENAI

Definição e utilização de componentes

- Definir um componente é criar uma função que determina o que será exibido na interface. Para utilizá-lo, basta chamá-lo com a sintaxe <NomeDoComponente />.
- Um mesmo componente pode ser usado várias vezes, permitindo combinar partes menores para construir páginas completas e organizadas.

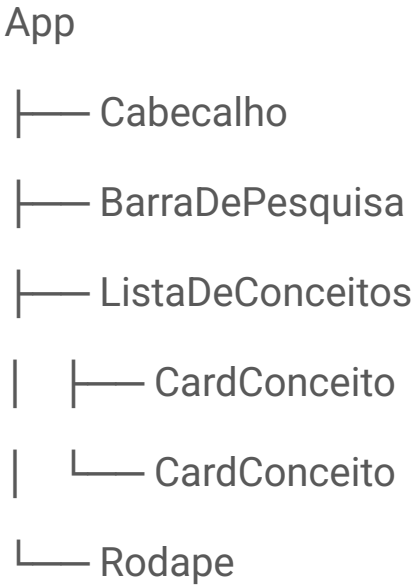
SENAI

Definição e utilização de componentes

```
function Rodape() {
  return <footer>Material de estudo</footer>;
}
function App() {
  return (
    <main>
      <h1>React Lab</h1>
      <Rodape />
    </main>
  );
}
```

Árvore de componentes

- Uma aplicação pode ser representada como uma árvore hierárquica.
- O componente principal contém componentes filhos.
- Os componentes filhos podem conter outros componentes.
- Essa estrutura ajuda a compreender a organização da interface.



SENAI

Componentes e reutilização

- Um componente pode ser reutilizado em diferentes partes da aplicação.
- A reutilização evita repetir estruturas semelhantes.
- Componentes ajudam a manter a consistência visual.
- Os dados específicos de cada utilização podem ser recebidos por meio de props.

SENAI

PROPS

O que são Props?

- Props é a abreviação de *properties* (propriedades).
- São dados recebidos por um componente React.
- Permitem enviar informações de um componente pai para um filho.
- Permitem reutilizar componentes com conteúdos diferentes.

SENAI

O que são Props?

```
function CardConceito({ titulo, descricao }) {
  return (
    <article>
      <h2>{titulo}</h2>
      <p>{descricao}</p>
    </article>
  );
}
```

Conceito-chave: a estrutura do componente permanece igual, enquanto os dados exibidos podem mudar.

SENAI

Por que utilizar Props?

- Permitem personalizar componentes reutilizáveis.
- Evitam duplicar estruturas de interface.
- Facilitam a passagem de informações entre componentes.
- Tornam explícitos os dados utilizados por cada componente.

Exemplo: um catálogo pode utilizar o mesmo componente de card para apresentar diferentes conceitos.

SENAI

Por que utilizar Props?

```
<CardConceito
  titulo="JSX"
  descricao="Sintaxe para descrever interfaces."
/>

<CardConceito
  titulo="Props"
  descricao="Entradas recebidas por componentes."
/>
```

SENAI

Por que utilizar Props?

```
function Saudacao({ nome }) {
  return <h1>Olá, {nome}!</h1>;
}
// Utilização
<Saudacao nome="Joao" />
<Saudacao nome="Maria" />
```

Resultado: cada utilização apresenta uma mensagem diferente, embora utilize o mesmo componente.

SENAI

Fluxo de dados entre componentes

- Em uma arquitetura comum, os dados fluem do componente pai para o filho.
- O componente pai organiza e fornece as informações.
- O componente filho recebe as props e apresenta os dados.
- Esse fluxo explícito facilita compreender de onde vêm as informações.

SENAI

Fluxo de dados entre componentes

```
function Card({ titulo }) {
  return <h2>{titulo}</h2>;
}
function App() {
  return (
    <main>
      <Card titulo="Componentes" />
      <Card titulo="Props" />
    </main>
  );
}
```

SENAI

Tipos de valores em Props

- Strings: textos informados entre aspas.
- Números: valores numéricos passados entre chaves.
- Booleanos: valores true ou false.
- Arrays e objetos: estruturas de dados que podem ser enviadas ao componente.
- Funções: referências a funções.
- Elementos React: conteúdo que pode ser renderizado pelo componente.

SENAI

VIRTUAL DOM

SENAI

O que é o DOM?

- DOM (Document Object Model) é a representação estruturada do documento HTML disponibilizada pelo navegador.
- Permite que o JavaScript consulte e modifique elementos da página.
- É possível alterar textos, atributos, estilos e elementos.
- Em interfaces complexas, gerenciar manualmente todas as alterações pode tornar o código difícil de manter.

```
document.querySelector("h1").textContent = "Olá!";
```

O que é o Virtual DOM?

- Virtual DOM é um termo utilizado para descrever uma representação da interface em memória.
- O React utiliza descrições de elementos para determinar como a interface deve se apresentar.
- O desenvolvedor descreve o resultado esperado em vez de indicar manualmente cada alteração visual.
- O React coordena o processo de atualização da interface.

DOM x Virtual DOM

- Representa a estrutura do documento no navegador.
- Pode ser consultado e modificado por JavaScript.
- Reflete a estrutura do documento renderizado.

- É um modelo conceitual para compreender as descrições da interface usadas pelo React.
- É utilizado pelo React para coordenar a atualização da interface.
- Ajuda a representar como a interface deve se apresentar.

Como funciona a atualização da interface?

- Mudança: ocorre uma interação ou atualização de dados.
- Renderização: o React executa novamente os componentes relevantes para calcular a nova descrição da interface.
- Reconciliação: o React compara a nova descrição com a anterior.
- Atualização: as alterações necessárias são aplicadas ao ambiente de renderização, como o DOM do navegador.

SENAI

Exemplo conceitual: contador

- Um contador começa exibindo o valor zero.
- Ao clicar em “Incrementar”, o estado passa a representar o valor um.
- A interface é descrita com base no valor atual do estado.
- O React coordena a atualização necessária para apresentar o novo número.
- Não é necessário localizar manualmente o elemento e substituir seu texto diretamente no DOM.

SENAI

Reconciliação e identidade dos elementos

- Reconciliação é o processo pelo qual o React determina as mudanças necessárias entre descrições sucessivas da interface.
- O React considera a estrutura dos elementos e sua identidade.
- Nem toda nova renderização exige alterações no DOM.
- O objetivo é coordenar a atualização da interface de acordo com as mudanças identificadas.

SENAI

Reconciliação e identidade dos elementos

- O React precisa identificar quais elementos representam a mesma parte lógica da interface.
- O tipo do elemento e sua posição na árvore ajudam nesse processo.
- Em listas, as propriedades key ajudam a identificar cada item.
- Chaves estáveis facilitam a associação entre itens quando a lista é alterada.

Exemplo: se o valor de um contador muda de zero para um, o React determina o que precisa mudar para exibir o novo valor.

SENAI

LISTAS E KEYS

SENAI

Por que renderizar listas no React?

- Aplicações frequentemente exibem listas de alunos, produtos, tarefas e conceitos. Esses dados normalmente são armazenados em arrays.
- O React permite transformar coleções de dados em elementos de interface.
- O método JavaScript map() permite gerar um elemento para cada item.
- Essa abordagem evita escrever manualmente cada elemento da lista.

SENAI

Por que renderizar listas no React?

Conceito-chave: map() transforma cada item de um array em um elemento da interface.

map() percorre os elementos de um array. Uma função é executada para cada item. O resultado é um novo array com os valores retornados.

```
const nome = ["Joao", "Maria", "Ana"];
const itens = conceitos.map((nome) => (
  <li>{nome}</li>
));
```

O que é key?

- key é uma propriedade especial utilizada pelo React para identificar elementos entre irmãos em uma lista.
- Ajuda o React a reconhecer quais itens foram mantidos, adicionados, removidos ou reordenados.
- Deve ser informada ao renderizar listas de elementos.
- Normalmente, utiliza-se um identificador associado a cada item dos dados.

SENAI

O que é key?

```
const alunos = [
  { id: 101, nome: "Ana" },
  { id: 102, nome: "Bruno" },
  { id: 103, nome: "Caio" }
];

{alunos.map((aluno) => (
  <li key={aluno.id}>{aluno.nome}</li>
))}
```

SENAI

Listas, filtros e atualização de dados

- Listas podem ser combinadas com métodos como filter() e map().
- filter() seleciona os itens que atendem a uma condição.
- map() transforma os itens selecionados em elementos JSX.
- As chaves devem continuar associadas aos mesmos itens após a filtragem.
- Quando uma lista está em estado, prefira produzir novos arrays em vez de modificar diretamente o array existente.

SENAI

Listas, filtros e atualização de dados

```
const conceitos = ["JSX", "Props", "State", "Hooks"];
function ConceitosBasicos() {
  const basicos = conceitos.filter(
    (item) => item !== "Hooks"
  );
  return (
    <ul>
      {basicos.map((item) => (
        <li key={item}>{item}</li>
      ))}
    </ul>
  );
}
```

SENAI

RENDERIZAÇÃO CONDICIONAL

SENAI

O que é renderização condicional?

- Permite mostrar, ocultar ou alterar partes da interface conforme uma condição.
- Utiliza expressões e operadores JavaScript dentro do JSX.
- Adapta a interface aos dados e ao estado da aplicação.

Exemplo: uma tela pode apresentar mensagens diferentes para usuários autenticados e não autenticados.

Utilizando if e return

- A instrução if permite executar diferentes caminhos conforme uma condição.
- É indicada quando as partes da interface são substancialmente diferentes.
- O return pode encerrar a execução do componente antecipadamente.
- Essa abordagem ajuda a manter o código organizado e legível.

Utilizando if e return

```
function MensagemAcesso({ autenticado }) {
  if (autenticado) {
    return <p>Bem-vindo de volta!</p>;
  }
  return <p>Entre para continuar.</p>;
}
```

Quando utilizar? Quando cada condição exige uma resposta ou interface diferente.

SENAI

Renderização condicional com dados e estado

- As condições podem depender de props, estado ou dados recebidos pela aplicação.
- Listas podem apresentar resultados ou uma mensagem quando estiverem vazias.
- Interfaces também podem representar estados de carregamento e erro.
- A condição deve refletir o estado real dos dados.

SENAI

Renderização condicional com dados e estado

```
function ListaResultados({ resultados }) {
  if (resultados.length === 0) {
    return <p>Nenhum resultado encontrado.</p>;
  }
  return (
    <ul>
      {resultados.map((resultado) => (
        <li key={resultado.id}>
          {resultado.nome}
        </li>
      ))}
    </ul>
  );
}
```

SENAI

Alternando mensagens de carregamento

- O operador ternário pode apresentar diferentes mensagens conforme o estado: *condicao ? valorSeVerdadeiro : valorSeFalso*
- É útil para alternar entre carregamento e resultado.
- A condição deve indicar claramente qual estado está ativo.
- Em aplicações maiores, carregamento, erro e sucesso podem precisar de tratamentos distintos.

Aplicação: catálogos, resultados de busca, listas de produtos e painéis administrativos.

SENAI

Alternando mensagens de carregamento

```
function ResultadoBusca({ carregando }) {
  return (
    <div>
      {carregando
        ? <p>Buscando...</p>
        : <p>Busca finalizada.</p>}
    </div>
  );
}
```

SENAI

Atividade

Desenvolvam uma aplicação React que utilize todos os conceitos estudados nesta aula, incluindo JSX, componentes, props, listas, renderização condicional e useState.

Criem um repositório público no GitHub e um README contendo a descrição, as funcionalidades, e os conceitos utilizados.

Referências

1. SOUZA, Natan. **Bootstrap 4: conheça a biblioteca front-end mais utilizada no mundo.** São Paulo: Casa do Código, 2018. E-book. Disponível em: https://plataforma.bvirtual.com.br.

2. MACHADO, Kheronn Khennedy. **Angular 11 e Firebase: construindo uma aplicação integrada com a plataforma do Google.** São Paulo: Casa do Código, 2021. E-book. Disponível em: https://plataforma.bvirtual.com.br. Acesso em: 13 maio 2025.

3. EIS, Diego. **Guia Front-end: o caminho das pedras para ser um dev front-end.** São Paulo: Casa do Código, 2015. E-book. Disponível em: https://plataforma.bvirtual.com.br.

4. GONÇALVES, Edson. **Desenvolvendo aplicações Web com JSP, Servlets, JavaServer Faces, Hibernate, EJB 3 Persistence e Ajax.** Rio de Janeiro: Ciência Moderna, c2007.

Referências

5. HARTCOPP, Patrícia Ferreira. **Métrica Web.** São Paulo: Contentus, 2020. E-book (94 p.). Disponível em: https://plataforma.bvirtual.com.br/Acervo/Publicacao/185191. Acesso em: 30 abr. 2024.

6. NIEDERAUER, Juliano. **Desenvolvendo Websites com PHP: aprenda a criar Websites dinâmicos e interativos com PHP e banco de dados.** 3. ed. São Paulo: Novatec, 2017.

7. PREECE, J.; ROGERS, Y.; SHARP, H. **Design de Interação: além da interação Homem-Computador.** 3. ed. Porto Alegre: Bookman, 2013.

8. SOUSA, Roque Fernando Marcos. **Canvas HTML 5: composição gráfica e interatividade na Web.** Rio de Janeiro: Brasport, 2014. E-book (194 p.). Disponível em: https://plataforma.bvirtual.com.br/Acervo/Publicacao/160686. Acesso em: 22 jun. 2024.

Projeto React

Prof. Me. Deivison S. Takatu

deivison.takatu@edu.senai.br